

VPA — Virtual Partner Architecture

Complete Learning Guide: From Beginner to Expert

Table of Contents

1. [Introduction](#)
 2. [Beginner Level: Understanding the Basics](#)
 3. [Intermediate Level: Deep Dive into Components](#)
 4. [Advanced Level: Customization and Extension](#)
 5. [Expert Level: System Architecture and Optimization](#)
 6. [Troubleshooting Guide](#)
 7. [API Reference](#)
 8. [Quick Reference Commands](#)
-

1. Introduction

1.1 What is VPA?

VPA (Virtual Partner Architecture) is a locally-running, multi-AI web system featuring two distinct AI personalities:

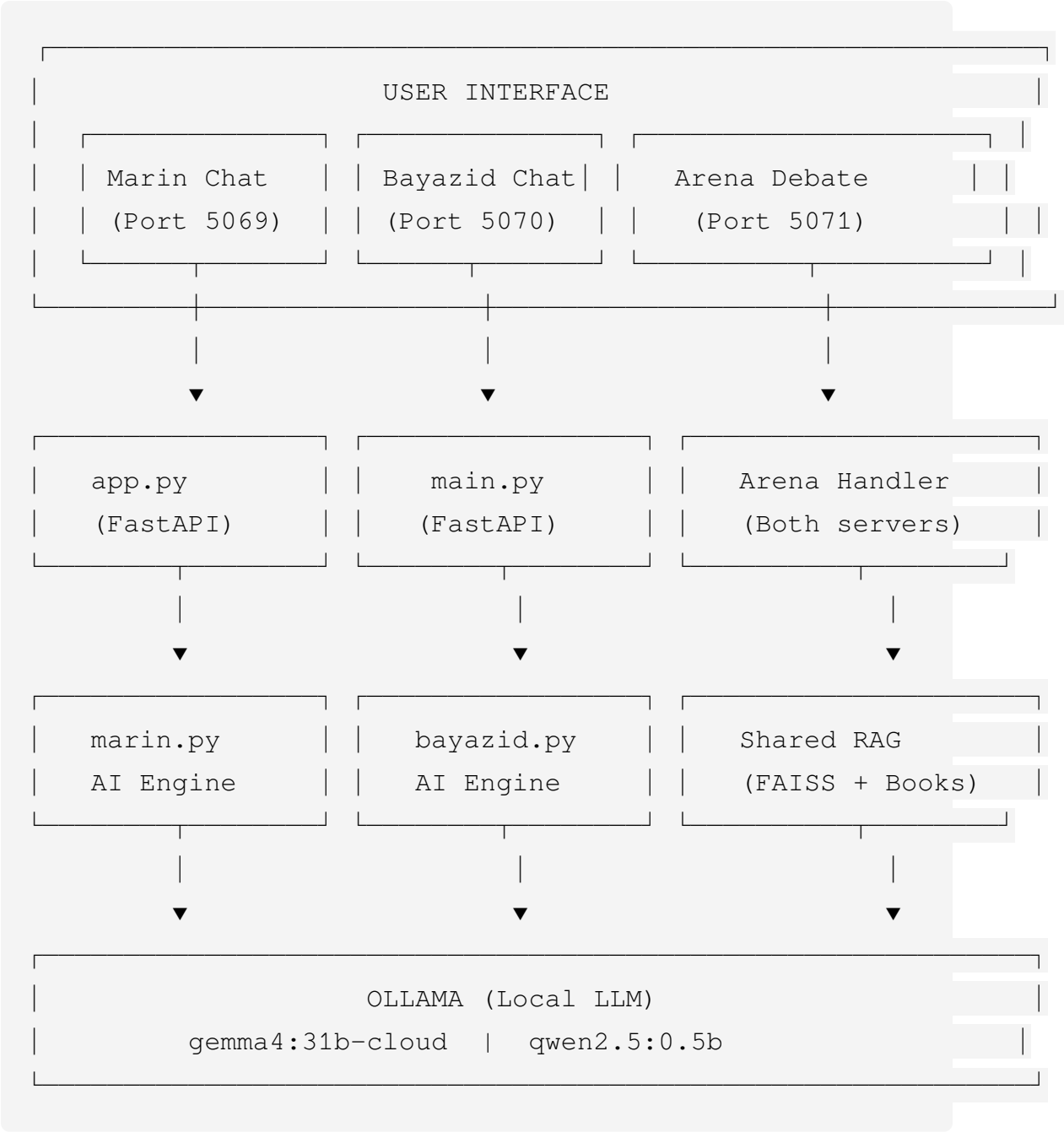
- **Marin Kitagawa** - A warm, emotionally intelligent AI companion
- **Bayazid HS-02** - A productivity-focused, analytical AI assistant

Both run on **Ollama** (local LLM inference) with no cloud dependencies, API keys, or subscriptions.

1.2 Core Philosophy

"Execution over illusion. Systems over chaos. Growth over dependency." —
Bayazid's motto "Always ♡" — Marin's motto

1.3 System Architecture Overview



2. Beginner Level: Understanding the Basics

2.1 Prerequisites

System Requirements

- Python 3.10 or higher
- [Ollama](#) installed and running
- At least 8GB RAM for model inference
- Linux/Windows/macOS

Installing Ollama

```
# Linux/macOS
curl -fsSL https://ollama.com/install.sh | sh

# Or use package manager
# Ubuntu/Debian: sudo apt install ollama

# Windows - Download from https://ollama.com
```

Pull Required Models

```
# Primary model (for main AI responses)
ollama pull gemma4:31b-cloud

# Fast model (for quizzes, games, quick tasks)
ollama pull qwen2.5:0.5b
```

2.2 Installation Steps

1. Clone or Download the Project

```
cd ~/projects
# If using git:
git clone <repository-url>
cd vpa
```

2. Install Python Dependencies

```
# Create virtual environment (recommended)
python -m venv .venv
source .venv/bin/activate # Linux/macOS
# or: .venv\Scripts\activate # Windows

# Install dependencies
pip install -r requirements.txt
```

3. Verify Installation

```
# Test Python imports
python -c "import fastapi; import ollama; print('OK')"
```



```
# Check Ollama is running
ollama list
```

2.3 Running the System

Quick Start Script

```
# Make scripts executable
chmod +x run_marin.sh run_bayazid.sh run_arena.sh
```

```
# Terminal 1 - Start RAG Server (optional, for book knowledge)
./rag_server.sh

# Terminal 2 - Start Marin
./run_marin.sh

# Terminal 3 - Start Bayazid
./run_bayazid.sh
```

Manual Start

```
# Terminal 1 - RAG Server
python rag_server.py

# Terminal 2 - Marin
python app.py

# Terminal 3 - Bayazid
python main.py
```

2.4 Accessing the Interfaces

Open your browser:

Interface	URL	Description
Marin Chat	http://localhost:5069	Marin's conversation interface
Bayazid Chat	http://localhost:5070	Bayazid's conversation interface
Arena	http://localhost:5071	Debate interface
Profile	http://localhost:5070/profile	User profile & stats

2.5 Basic Usage

Marin (Chat Mode)

- Type any message
- Upload images for analysis
- Play games: "Let's play Tic Tac Toe"
- Get learning cards: "Explain neural networks"

Bayazid (Chat Mode)

- Type any technical question
- Use modes: Chat / Teach / Quiz / Code Review / Study Plan
- Start timer: `/timer start [task]`
- Upload images of circuits/diagrams

3. Intermediate Level: Deep Dive into Components

3.1 The Two AI Engines

Marin AI Engine (`marin.py`)

Key Components:

```
# Character Configuration
BASE_CHARACTER = """
You are Marin (Limoni) – a high-performance strategic partner...
"""

# Vibe System - Detects user mood and adjusts response
VIBE_MODIFIERS = {
    "lovely": "Be extra affectionate and warm...",
    "flirty": "Playful romantic energy. Tease him lovingly...",
```

```

    "angry": "Be a bit cold, but still caring underneath...",
    "sad": "Be gentle, supportive, try to comfort him...",
    "excited": "Match his excitement, use more !!! and emojis...",
    "neutral": "",
}

```

Core Functions: - `main(prompt, image_path, game_context)` - Main async entry point - `response(prompt, user_vibe, ...)` - Generator for streaming responses - `classify(text)` - Intent + vibe detection - `load_history(limit)` - Load conversation history - `save_to_history(user_msg, marin_reply)` - Save to history

Bayazid AI Engine (`bayazid.py`)

Key Components:

```

# Character Configuration
BASE_CHARACTER = """
You are Bayazid HS-02 – a productivity-focused cognitive assistant...
"""

# Focus Timer
class StudyTimer:
    def start_session(self, task: str) -> None
    def end_session(self) -> Optional[Dict]
    def get_session_status(self) -> Dict
    def get_stats(self) -> Dict

# Conversation Memory
class ConversationMemory:
    def add(self, role: str, content: str) -> None
    def get(self) -> List[Dict]
    def clear(self) -> None

```

Core Functions: - `main(user_message, ...)` - Main chat - `teach_topic(topic, depth)` - Structured teaching - `generate_quiz(topic, difficulty, num_questions)` - Quiz

generation - `create_study_plan(topic, days)` - Study plans -
`review_code(code, language)` - Code review -
`explain_error(error, code)` - Error diagnosis

3.2 Intent Classification System

Marin Classifier (`marin_fier.py`)

Detects: - **Intent:** image_gen, play_tiktaktoe, play_connect4, play_wordgame, chat - **Vibe:** lovely, flirty, angry, sad, excited, playful, neutral, stressed, focused

Bayazid Classifier (`classifier.py`)

Detects: - **Intent:** chat, teach, quiz, study_plan, code_review, debug, code_gen, timer, vision, productivity - **Sub-intent:** Mode-specific (quick/standard/deep for teach, easy/medium/hard for quiz) - **Urgency:** normal, high

3.3 Memory System

How It Works

```
# Both AIs use similar memory patterns
history = load_history(limit=30) # Load last 30 messages
messages = [{"role": "system", "content": character_prompt}]
messages.extend(history)
messages.append({"role": "user", "content": user_message})

# Stream response from Ollama
for chunk in ollama.chat(model=MODEL, messages=messages, stream=True):
    yield chunk["message"]["content"]

# Save to history
save_to_history(user_message, response)
```


Storage

- **Marin:** `marin_history.json` (or MongoDB if available)
- **Bayazid:** `bayazid_history.json`

3.4 RAG System (Book Knowledge)

Setup

```
# Create doc directory
mkdir -p doc

# Copy PDF textbooks
cp ~/books/*.pdf doc/
```

How RAG Works

User Query → FAISS Search → Top K Chunks → Add to Prompt → AI Response

```
# In bayazid.py
book_context = rag.get_context_for_teaching(topic, k=20)

# Merged with system prompt:
messages = [
    {"role": "system", "content": BASE_CHARACTER + USER_CONTEXT + book_context},
    {"role": "user", "content": user_message}
]
```

Index Management

- **Automatic:** Index built on first server start
 - **Incremental:** New PDFs indexed automatically
 - **Manifest:** `faiss_db/manifest.json` tracks indexed files
-

4. Advanced Level: Customization and Extension

4.1 Customizing AI Personalities

Modifying Marin

Edit `marin.py` lines 53-84:

```
BASE_CHARACTER = """
You are Marin – [Your custom character description]
...
"""
```

Modifying Bayazid

Edit `bayazid.py` lines 38-88:

```
BASE_CHARACTER = """
You are Bayazid HS-02 – [Your custom character description]
...
"""
```

4.2 Changing Models

In Marin (`marin.py` line 41):

```
MODEL = "llama3.2:3b" # Change this line
```

In Bayazid (`bayazid.py` line 21):

```
MODEL = "mistral" # Change this line
```

Available Models

```
# List installed models
ollama list

# Pull new models
ollama pull phi3
ollama pull mistral
ollama pull deepseek-coder
```

4.3 Adding New Features

Adding a New Mode to Bayazid

```
# In bayazid.py
async def analyze_architecture(system_desc: str) -> AsyncIterator[str]:
    system_prompt = f"""{BASE_CHARACTER}

    [ARCHITECTURE ANALYSIS MODE]
    Analyze and design system architecture.

    Structure your response:
    ## Overview
    ## Components
    ## Data Flow
    ## Trade-offs
    """

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Analyze this system:\n{system_desc}"}
```

```

]

    async for chunk in _stream_model(messages):
        yield chunk

```

Adding to Classifier

```

# In classifier.py
intent_patterns = {
    # ... existing patterns ...
    "architecture": ["design architecture", "system design", "analyze
}

# Route in main.py
elif intent == "architecture":
    topic = extract_topic(message)
    return StreamingResponse(analyze_architecture(topic), media_type=

```

4.4 Adding Games

Game Structure

```

# games/tictaktoe.py
class TicTacToe:
    def __init__(self):
        self.board = {str(i): None for i in range(1, 10)}
        self.available = [str(i) for i in range(1, 10)]

    def make_move(self, cell: str, mark: str) -> bool:
        if cell in self.available:
            self.board[cell] = mark
            self.available.remove(cell)
            return True
        return False

```

```
def check_winner(self) -> Optional[str]:
    # Check rows, columns, diagonals
    pass
```

4.5 VSeeFace Avatar Integration

Setup

```
# expression.py
from python_osc import osc_message_builder

OSC_ADDRESS = "127.0.0.1"
OSC_PORT = 39539

EXPRESSIONS = {
    "fun": [("FaceAnimator/Fun", 1.0)],
    "joy": [("FaceAnimator/Joy", 1.0)],
    "neutral": [("FaceAnimator/Neutral", 1.0)],
    "sorrow": [("FaceAnimator/Sorrow", 1.0)],
    "angry": [("FaceAnimator/Angry", 1.0)],
}

def set_expression(name: str):
    # Reset all to 0, set target to 1
    for expr, value in EXPRESSIONS.get(name, []):
        send_osc(expr, value)
```

Run Independently

```
# Start expression handler (separate terminal)
python expression.py
```

5. Expert Level: System Architecture and Optimization

5.1 Architecture Decisions

Why Two Separate Servers?

- 1. **Independent Restarts:** Crashing one doesn't kill the other
- 2. **No Route Conflicts:** Different ports, no shared state
- 3. **Independent Scaling:** Can optimize each independently
- 4. **Ollama Queuing:** Handles multiple requests regardless

Why Single Model for Three Characters?

- One download, three personalities
- System prompts define character, not model weights
- Reduced resource usage
- Consistent context window management

Why FAISS for RAG?

- Fast similarity search
- Local, no cloud dependency
- Incremental indexing
- Proven at scale

5.2 Performance Optimization

Model Selection

Use Case	Model	Reason
Main Chat	gemma4:31b-cloud	Balance of speed and capability
Quiz Generation	qwen2.5:0.5b	Fast, sufficient for MCQ

Use Case	Model	Reason
Game AI	qwen2.5:0.5b	Needs only 1-2 tokens

Context Window Management

```
# Limit history to prevent context overflow
MEMORY_MAX_MESSAGES = 12 # Keep last 12 messages

# For teaching: include more context from books
k = {"quick": 10, "standard": 20, "deep": 30}.get(depth, 20)
```

Streaming Optimization

```
# FastAPI streaming response
return StreamingResponse(
    bayazid_main(message, image_path=image_path),
    media_type="text/plain"
)
```

5.3 Production Deployment

Using Systemd (Linux)

```
# /etc/systemd/system/marin.service
[Unit]
Description=Marin AI Server
After=network.target ollama.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/vpa
ExecStart=/usr/bin/python3 app.py
```

```
Restart=always

[Install]
WantedBy=multi-user.target
```

Using Docker

```
FROM python:3.10-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
EXPOSE 5069 5070

CMD ["sh", "-c", "python rag_server.py & python app.py & python main.py"]
```

5.4 Monitoring and Logging

Add Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
)

logger = logging.getLogger(__name__)

# Usage
logger.info("Processing request")
logger.error(f"Error: {e}")
```


Health Checks

```
# In main.py
@app.get("/health")
async def health():
    return {
        "status": "operational",
        "codename": "BAYAZID HS-02",
        "model": MODEL,
        "active_sessions": timer.get_stats()
    }
```

5.5 Security Considerations

API Security

```
# Add API key validation
from fastapi import Header

@app.post("/message")
async def handle_message(
    message: str = Form(...),
    api_key: str = Header(None)
):
    if api_key != os.environ.get("API_KEY"):
        return JSONResponse({"error": "Unauthorized"}, status_code=400)
    # ... rest of handler
```

Rate Limiting

```
from fastapi import Request
from slowapi import Limiter

limiter = Limiter(key_func=get_remote_address)
```

```
@app.post("/message")
@limiter.limit("10/minute")
async def handle_message(request: Request, ...):
    # ...
```

6. Troubleshooting Guide

6.1 Common Issues

Ollama Not Running

```
# Check if Ollama is running
ollama list

# Start Ollama service
ollama serve

# Check logs
journalctl -u ollama -f
```

Model Not Found

```
# Pull the model
ollama pull gemma4:31b-cloud

# Verify
ollama list
```

Port Already in Use

```
# Find process using port
lsof -i :5069
```

```
# Kill the process
kill -9 <PID>

# Or use different port in config.py
```

RAG Not Working

```
# Check doc directory
ls -la doc/

# Check FAISS index
ls -la faiss_db/

# Re-index
python -c "from bayazid import rag; rag.reindex_failed()"
```

Memory Errors

```
# Reduce context window
MEMORY_MAX_MESSAGES = 8 # Instead of 12

# Clear history
curl -X POST http://localhost:5070/memory/clear
```

6.2 Debug Mode

Enable Verbose Logging

```
# In app.py or main.py
import logging
logging.basicConfig(level=logging.DEBUG)
```

```
# Or set environment variable
export LOG_LEVEL=DEBUG
```

Test Classifier Directly

```
from classifier import classify

result = classify("Explain how PID controllers work")
print(result)
# Output: {'intent': 'teach', 'sub_intent': 'standard', 'confidence':
```

Test RAG Directly

```
from bayazid import rag

results = rag.search("PID controller", k=5)
for r in results:
    print(r["content"][:200])
```

7. API Reference

7.1 Bayazid Server (Port 5070)

Pages

Method	Path	Description
GET	/	Landing page
GET	/chat	Chat interface
GET	/profile	User profile & stats

Method	Path	Description
GET	/arena	Debate interface

Chat Endpoint

```
curl -X POST http://localhost:5070/message \  
  -F "message=Explain neural networks" \  
  -F "study_context={}"
```

Quiz Endpoint

```
curl -X POST http://localhost:5070/quiz/generate \  
  -F "topic=Python" \  
  -F "difficulty=medium" \  
  -F "num_questions=5"
```

Timer Endpoints

```
# Start timer  
curl -X POST http://localhost:5070/timer/start \  
  -F "task=ESP32 Project"  
  
# Stop timer  
curl -X POST http://localhost:5070/timer/stop  
  
# Get stats  
curl http://localhost:5070/timer/stats
```

Memory Endpoints

```
# Clear memory  
curl -X POST http://localhost:5070/memory/clear
```

```
# Get memory status
curl http://localhost:5070/memory/status
```

7.2 Marin Server (Port 5069)

Pages

Method	Path	Description
GET	/	Landing page
GET	/chat	Marin chat interface

Chat Endpoint

```
curl -X POST http://localhost:5069/message \
-F "message=Let's play tic tac toe"
```

Game Endpoints

```
# Start game
curl -X POST http://localhost:5069/game/start

# Make move
curl -X POST "http://localhost:5069/game/tiktaktoe/move?cell=5"

# Get AI move
curl -X POST http://localhost:5069/game/tiktaktoe/ai-move
```

8. Quick Reference Commands

8.1 Terminal Commands

```
# Start all servers
./run_marin.sh &      # Terminal 1
./run_bayazid.sh &    # Terminal 2

# Check processes
ps aux | grep python

# View logs
tail -f app.log

# Stop all
pkill -f "python app.py"
pkill -f "python main.py"
```

8.2 Timer Commands

Command	Description
/timer start [task]	Begin focus session
/timer stop	End current session
/timer status	Check current session
/timer stats	View productivity stats

8.3 Bayazid Modes

Mode	Usage
Chat	Default - any question
Teach	Use depth: "teach deep [topic]"
Quiz	"Generate quiz on [topic]"
Code Review	Paste code, ask for review
Study Plan	"Create study plan for [topic]"

8.4 Marin Commands

Command	Description
"Let's play Tic Tac Toe"	Start game
"Generate an image of..."	Image generation
"Explain [topic]"	Learning cards

8.5 File Locations

File	Purpose
<code>doc/</code>	PDF textbooks
<code>faiss_db/</code>	Vector index
<code>marin_history.json</code>	Marin memory
<code>bayazid_history.json</code>	Bayazid memory

File	Purpose
<code>timer_sessions.json</code>	Focus timer data
<code>vibe_state.json</code>	Current mood state

Appendix: Configuration Reference

config.py

```
# Server Configuration
HOST = "0.0.0.0"
PORT_MARIN = 5069
PORT_BAYAZID = 5070
PORT_RAG = 5080

# Model Configuration
DEFAULT_MODEL = "gemma4:31b-cloud"
FAST_MODEL = "qwen2.5:0.5b"

# Memory
MEMORY_MAX_MESSAGES = 12
HISTORY_FILE_MAX = 50

# RAG Configuration
DOC_DIR = "doc"
FAISS_DIR = "faiss_db"
EMBEDDING_MODEL = "all-MiniLM-L6-v2"
CHUNK_SIZE = 450
CHUNK_OVERLAP = 40
```

Last Updated: May 2026 Version: 1.0 For questions and support, refer to the project documentation.